

Unified Modeling Language

The **Unified Modeling Language (UML)** is a general-purpose, object-oriented, visual modeling language that provides a way to visualize the architecture and design of a system, similar to the function of a blueprint.^{[1][2]} UML defines notation for 14 types of diagrams which focus on aspects such as behavior, interaction, and structure.

The UML has a metamodel defined by the OMG UML specification. The metamodel defines the elements in an object-oriented model such as classes and properties. It is essentially the same thing as the metamodel in object-oriented programming (OOP), however for OOP, the metamodel is primarily used at run time to dynamically inspect and modify an application object model. The UML metamodel provides a mathematical, formal foundation for the graphic views used in the modeling language to describe an emerging system.

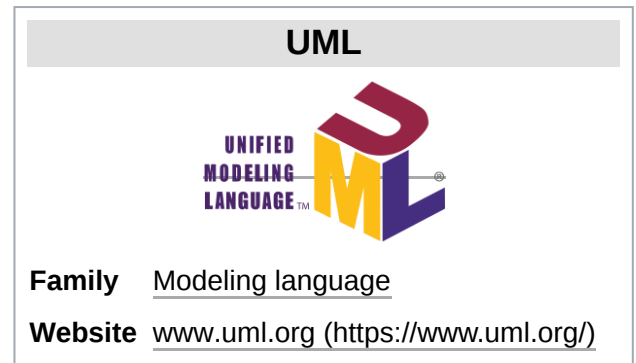
UML was created in an attempt to define a standard language for object-oriented programming at the OOPSLA '95 Conference. Originally, Grady Booch and James Rumbaugh merged their models into a unified model. This was followed by Booch's company Rational Software purchasing Ivar Jacobson's Objectory company and merging their model into the UML. At the time Rational and Objectory were two of the dominant players in the small world of independent vendors of object-oriented tools and methods.^[3] The Object Management Group (OMG) then took ownership of UML.

The creation of UML was motivated by the desire to standardize the disparate nature of notational systems and approaches to software design at the time.^[4] In 1997, UML was adopted as a standard by the Object Management Group (OMG) and has been managed by this organization ever since. In 2005, UML was also published by the International Organization for Standardization (ISO) and the International Electrotechnical Commission (IEC) as the **ISO/IEC 19501** standard.^[5] Since then the standard has been periodically revised to cover the latest revision of UML.^[6]

Most developers do not use UML per se, but instead produce more informal diagrams, often hand-drawn. These diagrams, however, often include elements from UML.^{[7]:536}

Use

UML is primarily used for software development (in any industry or domain)^[8] but also used outside elsewhere including business processes, system functions, database schemas, workflow in the legal systems, medical electronics, Health care systems, and hardware design.^[9] The UML is used by the OMG itself to define other OMG products such as the Unified Architecture Framework (UAF) and the Systems Modelling Language (SysML) v1.



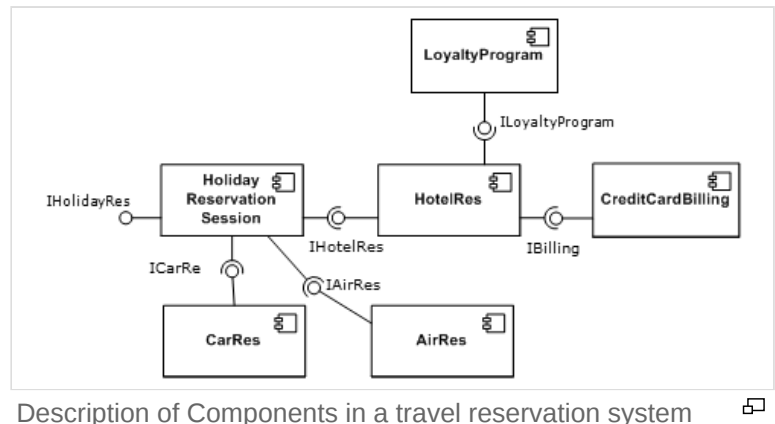
UML is designed for use with many object-oriented software development methods, both today and for the methods when it was first developed – including OMT, Booch method, Objectory, and especially RUP, which it was originally intended to be used with when work began at Rational Software.^[10] Although originally intended for object-oriented design documentation, UML has been used effectively in other contexts such as modeling business process.^{[11][12]}

As UML is not inherently linked to a particular programming language, it can be used for modeling a system independent of language. Some UML tools generate source code from a UML model.^[13]

Diagrams

UML diagrams support visualizing system aspects like:^[14]

- Use case diagram for specifying user interactions with systems
- Class diagram for specifying structures, including data structures
- Activity diagram for specifying business process workflows
- Component diagram for specifying how components interface with other components
- Deployment diagram for specifying how components are deployed and executed on computational nodes



In addition to syntactical (notational) elements with well-defined semantics, UML diagrams also allow for free-form comments (notes) that explain aspects such as usage, constraints, and intents.

Sharing

UML models can be exchanged among UML tools via the XML Metadata Interchange (XMI) format.

Cardinality notation

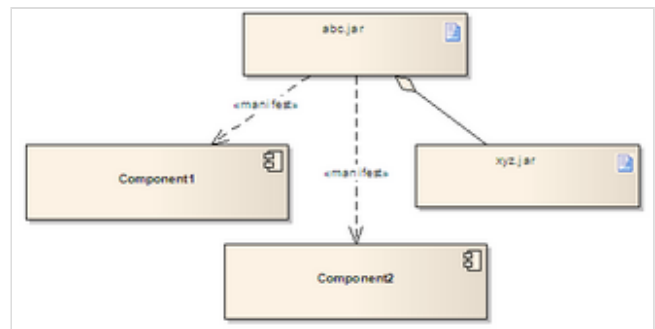
As with database Chen, Bachman, and ISO ER diagrams, class models are specified to use "look-across" cardinalities, even though several authors (Merise,^[15] Elmasri & Navathe,^[16] amongst others^[17]) prefer same-side or "look-here" for roles and both minimum and maximum cardinalities. Recent researchers (Feinerer^[18] and Dullea et al.^[19]) have shown that the "look-across" technique used by UML and ER diagrams is less effective and less coherent when applied to *n*-ary relationships of order strictly greater than 2.

Feinerer says: "Problems arise if we operate under the look-across semantics as used for UML associations. Hartmann^[20] investigates this situation and shows how and why different transformations fail.", and: "As we will see on the next few pages, the look-across interpretation introduces several difficulties which prevent the extension of simple mechanisms from binary to *n*-ary associations."

Artifacts

An artifact is the "specification of a physical piece of information that is used or produced by a software development process, or by deployment and operation of a system"^[21] including models, source code, scripts, executables, tables in database systems, development deliverables, design documents, and email messages.^[21]

An artifact is the physical entity that is deployed to a node.^[21] Other UML elements such as classes and components are first manifest into artifacts and instances of these artifacts are then deployed. Artifacts can be composed of other artifacts.



Artifact manifesting components

Metamodeling

The OMG developed a metamodeling architecture to define UML, called the Meta-Object Facility (MOF).^[22] MOF is designed as a four-layered architecture, as shown in the image at right. It provides a meta-meta model at the top, called the M3 layer. This M3-model is the language used by Meta-Object Facility to build metamodels, called M2-models.

An example of a Layer 2 Meta-Object Facility model is the UML metamodel, which describes the UML itself. These M2-models describe elements of the M1-layer, and thus M1-models. These would be, for example, models written in the UML. A software architecture description might describe runtime instances of the system.^[23] The last layer is the M0-layer - the real world thing being described or modelled.

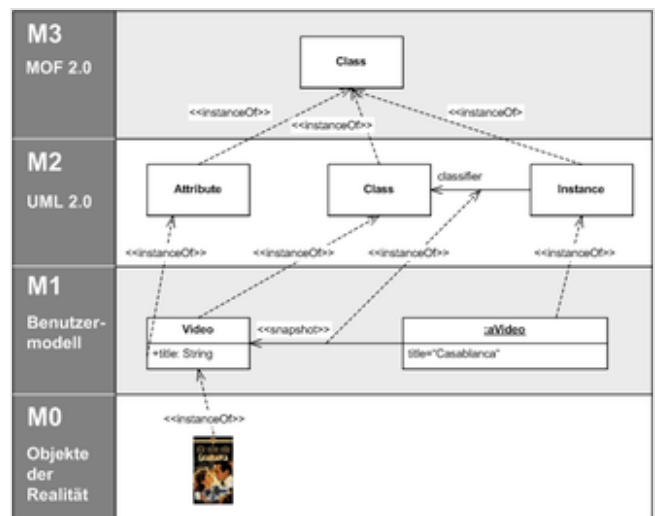
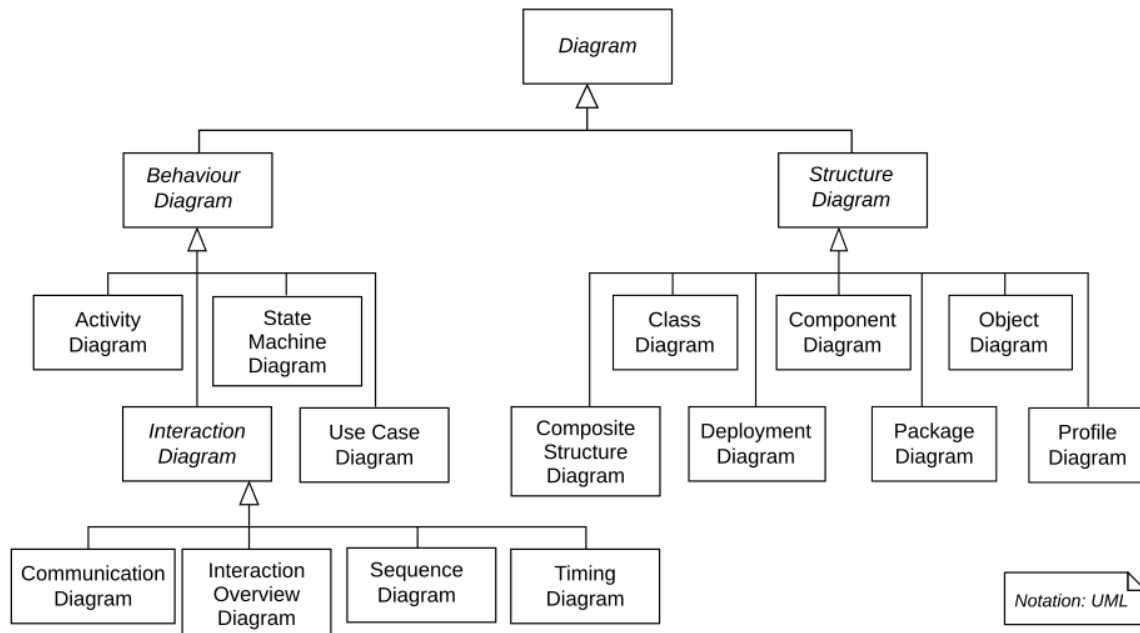


Illustration of the Meta-Object Facility

The UML metamodel can be extended using a mechanism called stereotyping. This has been criticized as being insufficient/untenable by Brian Henderson-Sellers and Cesar Gonzalez-Perez in "Uses and Abuses of the Stereotype Mechanism in UML 1.x and 2.0".^[24]

Diagrams

UML 2 defines 14 types or kinds of diagrams – shown as a taxonomy in the image.^{[14][25]}



Structure diagrams

Structure diagrams emphasize the structure of the system – using objects, classifiers, relationships, attributes and operations. They are used to document software architecture.

- Class diagram – Describes the structure of a class
- Component diagram – Describes how a software system is split into components and dependencies between the components
- Composite structure diagram
- Deployment diagram
- Object diagram
- Package diagram
- Profile diagram

Behavior diagrams

Behavior diagrams emphasize the behavior of a system by showing collaborations among objects and changes to the internal states of objects. They are used to describe the functionality of a system.

- Activity diagram – Describes the business and operational activities of components
- State machine diagram
- Use case diagram – Depicts of a user's interaction with a system^[26]

Interaction diagrams

Interaction diagrams, a subset of behavior diagrams, emphasize the flow of control and data between components of a system.

- Communication diagram – shows communication between components
- Interaction overview diagram

- Sequence diagram – shows interactions arranged in time sequence; can be drawn via tools such as Lucidchart and Draw.io
- Timing diagram – focuses on timing constraints

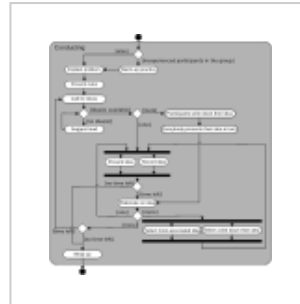
Examples



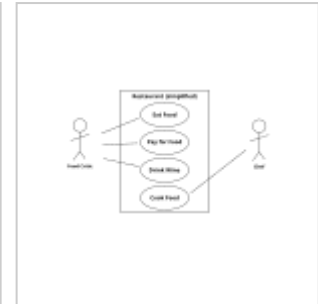
Component diagram



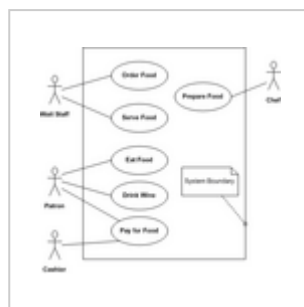
Class diagram



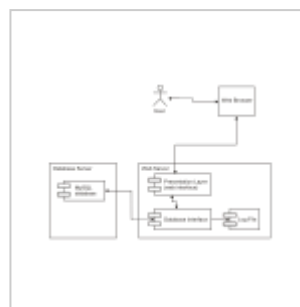
Activity diagram



Use case diagram



Use case diagram



Deployment diagram

Adoption

In 2013, UML had been marketed by OMG for many contexts, but aimed primarily at software development with limited success.^{[12][27]}

It has been treated, at times, as a design silver bullet, which leads to problems. UML misuse includes overuse (designing every part of the system with it, which is unnecessary) and assuming that novices can design with it.^[28]

It is considered a large language, with many constructs. Some people (including Jacobson) feel that UML's size hinders learning and therefore uptake.^[29]

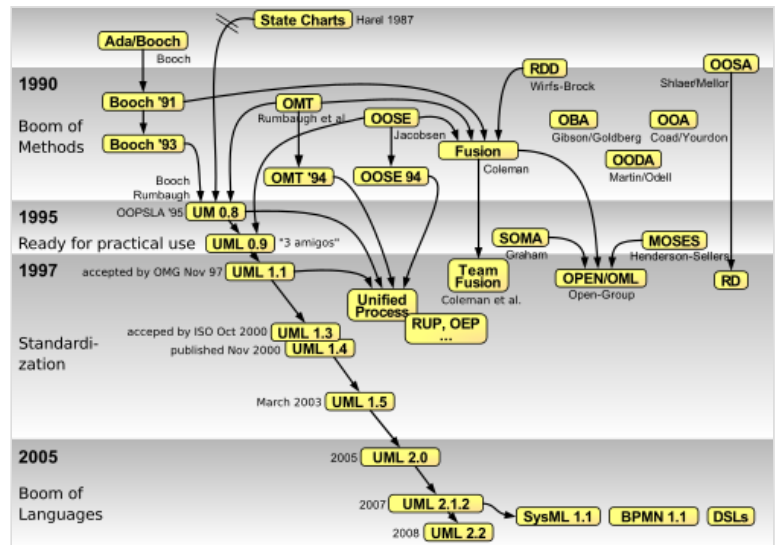
Visual Studio removed support for UML in 2016 due to lack of use.^[30]

History

UML has evolved since the second half of the 1990s and has its roots in the object-oriented programming methods developed in the late 1980s and early 1990s. The image shows a timeline of the history of UML and other object-oriented modeling methods and notation.

Origin

Rational Software hired James Rumbaugh from General Electric in 1994 and after that, the company became the source for two of the most popular object-oriented modeling approaches of the day:^[31] Rumbaugh's object-modeling technique (OMT) and Grady Booch's method. They were soon assisted in their efforts by Ivar Jacobson, the creator of the object-oriented software engineering (OOSE) method, who joined them at Rational in 1995.^[4]



Timeline and relationships of object-oriented methods and notation

UML 1.x

UML is originally based on the notations of the Booch method, the object-modeling technique (OMT), and object-oriented software engineering (OOSE), which were integrated into a single language.^[14] UML was developed at Rational Software in 1994–1995, with further development led by them through 1996.^[4]

Under the technical leadership of Rumbaugh, Jacobson, and Booch, a consortium called the UML Partners was organized in 1996 to complete the *Unified Modeling Language (UML)* specification and propose it to the Object Management Group (OMG) for standardization. The partnership also contained additional interested parties (for example HP, DEC, IBM, and Microsoft). The UML Partners' UML 1.0 draft was proposed to the OMG in January 1997 by the consortium. During the same month, the UML Partners formed a group, designed to define the exact meaning of language constructs, chaired by Cris Kobryn and administered by Ed Eykholt, to finalize the specification and integrate it with other standardization efforts. The result of this work, UML 1.1, was submitted to the OMG in August 1997 and adopted by the OMG in November 1997.^{[4][32]}

After the first release, a task force was formed^[4] to improve the language, which released several minor revisions, 1.3, 1.4, and 1.5.^[33]

The standards it produced (as well as the original standard) have been noted as being ambiguous and inconsistent.^[34]

UML 2

UML 2.0 major revision replaced version 1.5 in 2005, which was developed with an enlarged consortium to improve the language further to reflect new experiences on the usage of its features.^[35]

Although UML 2.1 was never released as a formal specification, versions 2.1.1 and 2.1.2 appeared in 2007, followed by UML 2.2 in February 2009. UML 2.3 was formally released in May 2010.^[36] UML 2.4.1 was formally released in August 2011.^[36] UML 2.5 was released in October 2012 as an "In progress" version and was officially released in June 2015.^[36] The formal version 2.5.1 was adopted in December 2017.^[1]

There are four parts to the UML 2.x specification:

- The Superstructure that defines the notation and semantics for diagrams and their model elements
- The Infrastructure that defines the core metamodel on which the Superstructure is based
- The Object Constraint Language (OCL) for defining rules for model elements
- The UML Diagram Interchange that defines how UML 2 diagram layouts are exchanged

Until UML 2.4.1, the latest versions of these standards were:^[37]

- UML Superstructure version 2.4.1
- UML Infrastructure version 2.4.1
- OCL version 2.3.1
- UML Diagram Interchange version 1.0.

Since version 2.5, the UML Specification has been simplified (without Superstructure and Infrastructure), and the latest versions of these standards are now:^[38]

- UML Specification 2.5.1
- OCL version 2.4

It continues to be updated and improved by the revision task force, who resolve any issues with the language.^[39]

See also

- Business Process Model and Notation – Graphical representation for specifying business processes
- C4 model – Technique for modelling software architecture
- Department of Defense Architecture Framework – Enterprise architecture framework
- DOT (graph description language) – File format
- List of Unified Modeling Language tools
- Real-Time UML – the application of the Unified Modelling Language (UML)
- MODAF – Enterprise architecture framework
- Model-based testing – Application of model-based design
- Model-driven engineering – Software development methodology
- Modeling and Analysis of Real Time and Embedded systems
- Object-oriented role analysis and modeling – Modeling method
- Process Specification Language – Set of logic terms used to describe processes
- Systems Modeling Language – General-purpose modeling language

References

1. *Unified Modeling Language 2.5.1* (<https://www.omg.org/spec/UML/2.5.1/PDF>). Object Management Group Document Number formal/2017-12-05. Object Management Group Standards Development Organization. December 2017.
2. [1412.2458] Systems, Views and Models of UML. By Ruth Breu Radu Grosu Franz Huber Bernhard Rumpe Wolfgang Schwerin. arXiv arxiv.org

3. "From Unified Method to Unified Language" (<http://modelseverywhere.wordpress.com/2010/11/13/from-unified-method-to-unified-language/>). *modelseverywhere.wordpress.com*. 13 November 2010. Retrieved 19 December 2013.
4. *Unified Modeling Language User Guide, The* (<http://www.informit.com/store/unified-modeling-language-user-guide-9780321267979>) (2 ed.). Addison-Wesley. 2005. p. 496. ISBN 0321267974. See the sample content: look for history
5. "ISO/IEC 19501:2005 - Information technology - Open Distributed Processing - Unified Modeling Language (UML) Version 1.4.3" (http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=32620). Iso.org. 1 April 2005. Retrieved 7 May 2015.
6. "ISO/IEC 19505-1:2012 - Information technology - Object Management Group Unified Modeling Language (OMG UML) - Part 1: Infrastructure" (http://www.iso.org/iso/iso_catalogue/catalogue_tc/catalogue_detail.htm?csnumber=32624). Iso.org. 20 April 2012. Retrieved 10 April 2014.
7. Sebastian Baltes; Stephan Diehl (11 November 2014). "Sketches and diagrams in practice" (<https://doi.org/10.1145/2635868.2635891>). *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. FSE 2014. Association for Computing Machinery. pp. 530–541. arXiv:1706.09172 (<https://arxiv.org/abs/1706.09172>). doi:10.1145/2635868.2635891 (<https://doi.org/10.1145%2F2635868.2635891>). ISBN 978-1-4503-3056-5. S2CID 2436333 (<https://api.semanticscholar.org/CorpusID:2436333>).
8. Conallen, Jim (1999). "Modeling Web Application Architectures with UML" (http://dl.acm.org/f_t_gateway.cfm?id=317677&type=pdf&CFID=36848136&CFTOKEN=65755783) (PDF). *Communications of the ACM*. **42** (10): 63–70. doi:10.1145/317665.317677 (<https://doi.org/10.1145%2F317665.317677>).
9. Jacobson, Ivar; Grady Booch; James Rumbaugh (1998). *The Unified Software Development Process*. Addison Wesley Longman. ISBN 0-201-57169-2.
10. John Hunt (2000). *The Unified Process for Practitioners: Object-oriented Design, UML and Java*. Springer, 2000. ISBN 1-85233-275-1. p. 5.door
11. Satish Mishra (1997). "Visual Modeling & Unified Modeling Language (UML): Introduction to UML" (http://www2.informatik.hu-berlin.de/~hs/Lehre/2004-WS_SWQS/20050107_Ex_UML.ppt) Archived (https://web.archive.org/web/20110720091651/http://www2.informatik.hu-berlin.de/~hs/Lehre/2004-WS_SWQS/20050107_Ex_UML.ppt) 20 July 2011 at the Wayback Machine. Rational Software Corporation. Accessed 9 November 2008.
12. "UML, Success Stories" (http://www.uml.org/uml_success_stories/index.htm). Retrieved 9 April 2014.
13. "Importance of UML" (<http://www.coderanch.com/t/98273/patterns/importance-UML>).
14. "OMG Unified Modeling Language (OMG UML), Superstructure. Version 2.4.1" (<http://www.omg.org/spec/UML/2.4.1/Superstructure/PDF>). Object Management Group. Retrieved 9 April 2014.
15. Hubert Tardieu, Arnold Rochfeld and René Colletti La methode MERISE: Principes et outils (Paperback - 1983)
16. Elmasri, Ramez, B. Shamkant, Navathe, Fundamentals of Database Systems, third ed., Addison-Wesley, Menlo Park, CA, USA, 2000.
17. Paolo Atzeni; Wesley Chu; Hongjun Lu; Shuigeng Zhou; Tok Wang Ling, eds. (27 October 2004). *Conceptual Modeling – ER 2004: 23rd International Conference on Conceptual Modeling, Shanghai, China, November 8–12, 2004*. *Lecture Notes in Computer Science* 3288 (2004 ed.). Springer. ISBN 3540237232.
18. Ingo Feinerer (March 2007). *A Formal Treatment of UML Class Diagrams as an Efficient Method for Configuration Management* (https://web.archive.org/web/20240629135535/http://publik.tuwien.ac.at/files/pub-inf_4582.pdf) (PDF) (Doctor of Technical Sciences thesis). Vienna: Technical University of Vienna. Archived from the original (https://publik.tuwien.ac.at/files/pub-inf_4582.pdf) (PDF) on 29 June 2024. Retrieved 25 May 2024.

19. James Dullea; Il-Yeol Song; Ioanna Lamprou (1 November 2003). "An analysis of structural validity in entity-relationship modeling". *Data & Knowledge Engineering*. **47** (2): 167–205. doi:10.1016/S0169-023X(03)00049-1 (<https://doi.org/10.1016%2FS0169-023X%2803%2900049-1>).
20. Sven Hartmann (17 January 2003). *Reasoning about participation constraints and Chen's constraints* (<https://dl.acm.org/doi/abs/10.5555/820085.820110>). ADC '03: Proceedings of the 14th Australasian database conference. Australian Computer Society. pp. 105–113. .
21. "Artifacts". *Unified Modeling Language 2.5.1* (<https://www.omg.org/spec/UML/2.5.1/PDF>). OMG Document Number formal/2017-12-05. Object Management Group Standards Development Organization (OMG SDO). December 2017. p. 656.
22. Iman Poernomo (2006) "The Meta-Object Facility Typed (<http://calcium.dcs.kcl.ac.uk/1259/1/acm-paper.pdf>) Archived (<https://web.archive.org/web/20160630002118/http://calcium.dcs.kcl.ac.uk/1259/1/acm-paper.pdf>) 30 June 2016 at the Wayback Machine" in: *Proceeding SAC '06 Proceedings of the 2006 ACM symposium on Applied computing*. pp. 1845–1849
23. "UML 2.4.1 Infrastructure" (<http://www.omg.org/spec/UML/2.4.1/Infrastructure/PDF/>). Omg.org. 5 August 2011. Retrieved 10 April 2014.
24. Brian Henderson-Sellers; Cesar Gonzalez-Perez (1 October 2006). "Uses and abuses of the stereotype mechanism in UML 1.x and 2.0". *MoDELS '06: Proceedings of the 9th International Conference on Model Driven Engineering Languages and Systems*. Lecture Notes in Computer Science 4199. **4199**. Berlin, Germany: Springer-Verlag: 16–26. doi:10.1007/11880240_2 (https://doi.org/10.1007%2F11880240_2). ISBN 978-3-540-45772-5.
25. Jon Holt Institution of Electrical Engineers (2004). *UML for Systems Engineering: Watching the Wheels* IET, 2004, ISBN 0-86341-354-4. p. 58
26. Manuel Almendros-Jiménez, Jesús & Iribarne, Luis. (2007). Describing Use-Case Relationships with Sequence Diagrams. *Comput. J.*. 50. 116-128. 10.1093/comjnl/bxl053.
27. "UML 2.5: Do you even care?" (<http://www.drdobbs.com/architecture-and-design/uml-25-do-you-even-care/240163702?queryText=uml>). *Dr. Dobb's*. "UML truly is ubiquitous"
28. "Death by UML Fever" (<http://queue.acm.org/detail.cfm?id=984495>).
29. "Ivar Jacobson on UML, MDA, and the future of methodologies" (http://www.infoq.com/interviews/Ivar_Jacobson).
30. Krill, Paul (18 October 2016). "UML to be ejected from Microsoft Visual Studio" (<https://www.infoworld.com/article/3131600/uml-to-be-ejected-from-microsoft-visual-studio.html>). *InfoWorld*. Retrieved 5 January 2026.
31. Andreas Zendler (1997) *Advanced Concepts, Life Cycle Models and Tools for Object-Oriented Software Development*. p. 122
32. "UML Specification version 1.1 (OMG document ad/97-08-11)" (<http://www.omg.org/cgi-bin/doc?ad/97-08-11>). Omg.org. Retrieved 22 September 2011.
33. "UML" (<http://www.omg.org/spec/UML/>). Omg.org. Retrieved 10 April 2014.
34. Génova et alia 2004 "Open Issues in Industrial Use Case Modeling"
35. "UML 2.0" (<http://www.omg.org/spec/UML/2.0/>). Omg.org. Retrieved 22 September 2011.
36. "UML" (<http://www.omg.org/spec/UML/>). Omg.org. Retrieved 22 September 2011.
37. OMG. "OMG Formal Specifications (Modeling and Metadata paragraph)" (<http://www.omg.org/spec/#M&M>). Retrieved 12 February 2016.
38. OMG. "about the unified modeling language specification" (<https://www.omg.org/spec/UML/About-UML/>). Retrieved 22 February 2020.
39. "Issues for UML 2.6 Revision task Force mailing list" (<https://issues.omg.org/issues/lists/uml2-rtf>). Omg.org. Retrieved 10 April 2014.

Further reading

- Tegarden, David et al. (2025). *Systems Analysis and Design: An Object-Oriented Approach with UML* (<https://www.wiley.com/en-us/Systems+Analysis+and+Design%3A+An+Object-Oriented+Approach+with+UML%2C+7th+Edition-p-9781394331727>) (7th ed.). Wiley. ISBN 978-1-394-33172-7 (<https://www.wiley.com/en-us/Systems+Analysis+and+Design%3A+An+Object-Oriented+Approach+with+UML%2C+7th+Edition-p-9781394331727>).
- Ambler, Scott William (2004). *The Object Primer: Agile Model Driven Development with UML 2* (<https://web.archive.org/web/20100131215505/http://www.ambysoft.com/books/theObjectPrimer.html>). Cambridge University Press. ISBN 0-521-54018-6. Archived from the original (<http://www.ambysoft.com/books/theObjectPrimer.html>) on 31 January 2010. Retrieved 29 April 2006.
- Chonoles, Michael Jesse; James A. Schardt (2003). *UML 2 for Dummies* (<https://archive.org/details/uml2fordummies00chon>). Wiley Publishing. ISBN 0-7645-2614-6.
- Fowler, Martin (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley. ISBN 0-321-19368-7.
- Jacobson, Ivar; Grady Booch; James Rumbaugh (1998). *The Unified Software Development Process*. Addison Wesley Longman. ISBN 0-201-57169-2.
- Martin, Robert Cecil (2003). *UML for Java Programmers* (<https://archive.org/details/umlforjavaprogram00mart>). Prentice Hall. ISBN 0-13-142848-9.
- Noran, Ovidiu S. "Business Modelling: UML vs. IDEF" (<https://web.archive.org/web/20240718044149/https://www.area-c54.it/public/business%20modelling%20-%20uml%20vs%20idef.pdf>) (PDF). Archived from the original (<https://www.area-c54.it/public/business%20modelling%20-%20uml%20vs%20idef.pdf>) (PDF) on 18 July 2024. Retrieved 14 November 2022.
- Horst Kargl. "Interactive UML Metamodel with additional Examples" (<http://umlnotation.sparxsystems.eu/>).
- Penker, Magnus; Hans-Erik Eriksson (2000). *Business Modeling with UML* (<https://archive.org/details/businessmodeling00erik>). John Wiley & Sons. ISBN 0-471-29551-5.
- Douglass, Bruce Powel. "Bruce Douglass: Real-Time Agile Systems and Software Development" (<https://www.bruce-douglass.com/>) (web). Retrieved 1 January 2019.
- Douglass, Bruce (2014). *Real-Time UML Workshop 2nd Edition* (<https://archive.org/details/businessmodeling00erik>). Newnes. ISBN 978-0-471-29551-8.
- Douglass, Bruce (2004). *Real-Time UML 3rd Edition*. Newnes. ISBN 978-0321160768.
- Douglass, Bruce (2002). *Real-Time Design Patterns*. Addison-Wesley Professional. ISBN 978-0201699562.
- Douglass, Bruce (2009). *Real-Time Agility*. Addison-Wesley Professional. ISBN 978-0321545497.
- Douglass, Bruce (2010). *Design Patterns for Embedded Systems in C*. Newnes. ISBN 978-1856177078.

External links

- Official website (<https://www.uml.org/>) 
- Current UML specification: *Unified Modeling Language 2.5.1* (<https://www.omg.org/spec/UM/L2.5.1/PDF>). OMG Document Number formal/2017-12-05. Object Management Group Standards Development Organization (OMG SDO). December 2017.

